

Freedom Scientific Scripting Standards

This document contains a set of standards which is required for all script submissions to Freedom Scientific. Since failure to conform to these standards results in scripts which are either buggy, suffer from degraded performance, or which are not localizable to multiple languages, all submissions of scripts must conform to the following standards in order to be considered for acceptance:

1. Critical: Do not break any currently existing features or intended functionality!
2. In the JSH file, define all script constants and globals. The only exception to this is for constants or globals that must be strictly local to the current script file and that should not be exposed to other script files.
3. In the JSM file, define all messages to be spoken, and use only those message names in the JSS file.
4. In the JSM file, define all strings used for comparison or for identification as constants, and use only those string names in the JSS file.
5. In the JSM file, add a comment for string names whose meaning is not obvious, explaining to translators where to find the text those string names represent in the comparison, or where to use those string names for identification in the program.
6. In the JSM file, define all window names and classes as constants, and use only those constants in the JSS file.
7. In the JSM file, define all string variables that are messages consisting of a specific formatted pattern such that each variable uses a variable placeholder in the formatted message. Then use the `FormatString` or `SayFormattedMessage` functions in the JSS file to format the message.
8. In the JSM file, document all replaceable parameters for each message that has them with comments explaining what information goes into the replaceable parameters.
9. Name all defined constants and variables for windows or controls so that they are self-documenting in order that a scripter or localizer can recognize the location of the window or control. If it belongs to a dialog, include the dialog name as part of the constant or variable name. If it belongs to a page in a multi-page dialog, include the page name as part of the constant or variable name. If it does not belong to a dialog, name the constant or variable so that the scripter or localizer can determine to which view or component of the program the window or control belongs.
10. Create different constant or variable names for windows or controls that have the same names but appear in more than one area or in unrelated areas of the program. In this way, windows or controls each have their own constant or variable names that refer specifically to where they are used. Do not re-use the same constant or variable name to refer to different windows or controls.
11. In the JSD documentation file, document all your functions and scripts. Take care to document well the synopsis, description, and any parameters or return types for each function and script. For scripts in particular, Be sure that the text of the Synopsis and Description fields is appropriate to be spoken and displayed in Braille by the JAWS Keyboard Help feature.
12. Do not attach the SPACEBAR or any alphanumeric keys to scripts. If special behavior is required for these keys, use the `KeyPressedEvent` function instead of attaching scripts to the keys.
13. Use the `SayString` function only as a debugging tool during scripting, and do not use this function to speak anything in the finished product.
14. Do not use braces like "{" and "}" to simulate keystrokes. Use the `TypeKey` or `TypeCurrentScriptKey` functions when sending a keystroke to the application.
15. Structure your code so that it tests and executes in the most expedient manner

possible. When adding code to frequently-run events, be especially careful not to introduce unnecessary sluggishness.

16. Use window or object information as opposed to screen information where possible.
17. Use the JAWS or Invisible cursor to obtain information only when no other method works. In this case, always use the Invisible cursor unless the JAWS cursor is required to perform the action because the Invisible cursor cannot. If it is necessary to use the JAWS or Invisible cursor, store the location of the PC cursor prior to activating the JAWS or Invisible cursor, or prior to calling a function that activates either of these cursors. Then take care to re-activate the PC cursor at the stored location as soon as the desired action is completed with the JAWS or Invisible cursor.

Note: As of JAWS 2020, you might instead choose to use the UIA Scan cursors. See the functions related to these cursors available for JAWS 2020 and later. Generally speaking, the same rules apply to the use of these cursors as to the traditional JAWS and Invisible cursors.

There are several methods to work with these cursors.

18. Never use the SpeechOff and SpeechOn functions as a means of suppressing speech output temporarily. If you must suppress speech for specific purposes, use a specific global variable for the specific suppression.
19. Always respect the active cursor when defining behavior in a script or function. Do not break the functionality of the JAWS, Invisible cursor, or the UIA Scan cursors.
20. When designing your code, always consider whether the desired JAWS behavior should honor the status of the user buffer or the menu state as special or exception cases.
21. When designing your code, always respect the user's speech output configuration, and always honor speech and sound scheme functionality for controls and for text properties.
22. Keep all global variables up to date. If you overwrite a function that has global variables, be sure to update the necessary variables before returning from that function.
23. Overwrite for the specific circumstance when overwriting the behavior of an existing function or script. Then call the default function or script for all other circumstances. Wherever possible, avoid copying the entire default function or script to your script file and then adding exception cases. Doing so renders your code obsolete if the default code changes. Always use the same name for your function or script as the name of the default function or script.
24. Use event-driven code where appropriate. For instance, in lists or trees where first-letter navigation is possible, make sure that any code for outputting speech during navigation uses appropriate event-driven code such as the SayHighlightedText function (called by the NewTextEvent function), the ActivItemChangedEvent or ValueChangedEvent function instead of key-driven code like the SayNextLine or SayPriorLine scripts.
25. When designing your code for overwriting the functionality of a keystroke native to the application, evaluate whether the keystroke should be passed through to the application if the special testing conditions in your code are not met. In the JSM file, define the key name to be passed through as a constant with a meaningful name that explains the keystroke's functionality. For example: Do not use ksAltI; rather, use ksIgnoreButton since the hotkey for the button may vary from language to language.
26. If overwriting the functionality of a key combination already defined in the default JKM file, do not make any entry in the application-specific JKM file for the key combination.
27. In JAWS hotkey help, include all new non-application and non-standard Windows key maps for all keys added to the JKM file.

28. Ensure that the ScriptFileName function outputs relevant information for the current application and script file.
29. Do not output any special messages for scripts attached to native Windows keystrokes.
30. Ensure that the appropriate information (Application window information, real window information, and focus window information) is spoken when focus changes. Eliminate any double-speaking, and ensure that only relevant changes are spoken.
31. Ensure that the SayWindowPromptAndText function speaks the same information about the focus window that was spoken when the window gained focus. In addition, toggle on training mode temporarily for the SayWindowPromptAndText function when speaking the focus window so that training information is spoken as appropriate.
32. When designing help messages to be shown in the virtual viewer, place a statement at the top of the virtual viewer telling the user the current location. If showing a list of commands for a key layer, show what key should be pressed to start the layer. If showing a list of commands and their associated keystrokes, make each command in the list a link. Then for each command link, describe the command first, and then show the keystroke. Show any more general information in paragraph form without any links. Always end each virtual help screen with a message telling the user how to close the virtual viewer. This message need not be a link.
33. When scripting for special behavior, script both for speech and for Braille behaviors.
34. On controls where the position in group information is available, make sure that the position in the group is announced when focus moves to the control. Also, ensure that your special conditions cause the SayWindowPromptAndText and SayLine functions to announce the position in group. However, also ensure that JAWS does not announce the position in group when moving between the items in the group. JAWS should announce position in group information for controls such as lists, combo boxes, and radio buttons. For treeviews, along with position in group information, JAWS should also announce level and state information. Ensure that JAWS announces and displays in Braille the position, level and open/close state changes of treeview items as they are navigated.

Additional Guidelines for Writing Acceptable Scripting Code

Following are some additional guidelines for writing and formatting code so that it can be most easily maintained. The development staff at Freedom Scientific thanks you very much for following these guidelines when writing your code.

- Write self-documenting code by using meaningful script, function, variable and constant names. Self-documenting code is much easier to understand, maintain and debug.
- Modularize code so that your scripts or functions do not become large and unwieldy. It is much more time-efficient to maintain and debug modular, smaller scripts and functions than to step through a large section of code line-by-line.
- Capitalize script, function, variable and constant names so that JAWS speaks them in a recognizable manner. This is best accomplished either by capitalizing the individual "words" of the name, or by using underscore ("_") to separate the component "words" of the name.
- Since the compiler does not allow for nested C-style comments, do not use C-style comments ("/*" "*/") to comment code. Instead, use semicolon (";"). This will make it easier to use C-style comments as a temporary means to comment out code since the scripter will not need to look for existing C-style comment markers inside of the code.
- When defining local variables, place "var" on a line by itself at the left margin, and

then place each succeeding local variable on a line by itself indented one tab stop. This makes the variable list easy to read quickly and to modify if items need to be added or deleted.

- Place all level-0 statements at the left margin, and indent once using tab (not spaces) for each level. Line up the matching components of If-Elif-Else-EndIf statements and of While-EndWhile statements. When an If or While statement contains compound conditions, place each condition on a line by itself. Line up the operator for each line of the compound test with the If, the Elif, the Else, and the Endif, or the While-Endwhile statement. Following this convention makes it easy to find code blocks displayed in Braille and by using the JAWS indentation announcement capabilities.
- Comment your code judiciously. If the code itself is self-documenting, comments are unnecessary. However, if you are coding in a specific manner or employing a technique whose purpose may not be readily apparent simply by reading the code, then comments are critical. Remember that your code and comments may be read by anyone. So please keep the comments professional and appropriate for all readers.

Naming Conventions for Variables and Constants

Use a standard and consistent method for naming variables and constants. The Freedom Scientific scripting code adheres mostly to the following naming conventions for notating types of variables and constants:

- Precede handle variable names with "h", string variable names with "s", integer variable names with "i" and object variable names with "o".
- Precede global variable names with "g". For example, a global integer variable name is preceded with "gi".
- When naming constants, precede window names with "wn", and window classes with "wc".
- When defining messages to be spoken, precede the name with "msg"; use "sc" as a prefix to constant strings defined for on-screen comparison.
- Be consistent with your convention, and choose a meaningful convention if an appropriate one does not already exist. Using this type of notation makes it easier to know instantly what type of variable is being used without having to look back at the declaration list in JSH or JSM files.